Document Information	
Filename	TP_F101U-00054_User_Guide_SecureBoot_5G_NAD_ConMod
Status	Release
Revision Date	Aug 21, 2024
Version	02.00
Document ID	TP_F101U-00054

Standard Proprietary Terms

THE REPRODUCTION, DISTRIBUTION AND UTILIZATION OF THIS DOCUMENT AS WELL AS THE COMMUNICATION OF ITS CONTENTS TO OTHERS WITHOUT EXPRESS AUTHORIZATION IS PROHIBITED. OFFENDERS WILL BE HELD LIABLE FOR THE PAYMENT OF DAMAGES. ALL RIGHTS RESERVED IN THE EVENT OF THE GRANT OF A PATENT, UTILITY MODEL OR DESIGN.

© Continental Automotive GmbH 2020
Confidential and Proprietary

Table of contents

1 Introduction 4
1.1 Notational Conventions 4
2 Setup 4
2.1 Tools/Equipment Needed 4
2.2 Setup Instructions 4
2.3 Setup Troubleshooting 5
3 Features/Components 5
3.1 Image signing 5
3.1.1 Overview 5
3.1.1.1 Platform Content Overview 5
3.1.1.2 Product Expectations 6
3.1.1.3 Extending Platform Functionality 6
3.1.2 Setup 6
3.1.2.1 Tools/Equipment Needed 6
3.1.2.2 Setup Instructions 6
3.1.2.3 Setup Troubleshooting 6
3.1.3 Usage 7
3.1.3.1 Basic Instructions 7
3.1.3.2 Detailed Instructions 7
3.1.3.3 Testing/Error Reporting 8
3.2 Enable Secure Boot 8
3.2.1 Overview 8
3.2.1.1 Platform Content Overview 8
3.2.1.2 Product Expectations 8
3.2.1.3 Extending Platform Functionality 8
3.2.2 Setup 8
3.2.2.1 Tools/Equipment Needed 8
3.2.2.2 Setup Instructions 8
3.2.2.3 Setup Troubleshooting 8
3.2.3 Usage 8
3.2.3.1 Basic Instructions 8
3.2.3.2 Detailed Instructions 8
3.2.3.3 Testing/Error Reporting 9
3.3 Suspend Secure Boot 9
3.3.1 Overview 9
3.3.1.1 Platform Content Overview 9
3.3.1.2 Product Expectations 9
3.3.2 Setup 9
3.3.2.1 Tools/Equipment Needed 9

© Continental Automotive GmbH 2020
Confidential and Proprietary

3.3.2.2 Setup Instructions 9
3.3.2.3 Setup Troubleshooting 9
3.3.3 Usage 9
3.3.3.1 Basic Instructions 9
3.3.3.2 Detailed Instructions 10
3.3.3.3 Testing/Error Reporting 10
3.4 Re-enabling of ram dump, JTAG and Trusted logs 10
3.4.1 Overview 10
3.4.1.1 Platform Content Overview 10
3.4.1.2 Product Expectations 10
3.4.1.3 Extending Platform Functionality 11
3.4.2 Setup 11
3.4.2.1 Tools/Equipment Needed 11
3.4.2.2 Setup Instructions 11
3.4.2.3 Setup Troubleshooting 11
3.4.3 Usage 11
3.4.3.1 Basic Instructions 11
3.4.3.2 Detailed Instructions 11
3.4.3.3 Testing/Error Reporting 13
4 Known Issues 13
5 FAQs 13
6 Troubleshooting/Tips and Tricks 13
7 References 13
7.1 Abbreviations/Acronyms/Definitions 13
7.2 Platform Documentation 13
7.3 Website links 13
8 Revision History 14

© Continental Automotive GmbH 2020
Confidential and Proprietary

1 Introduction

Secure boot provides a foundation for the security architecture of the device. Technically, secure boot is defined as a boot sequence in which each software image that is loaded and executed on a device is authorized using software previously authorized by this system. This sequence is designed to prevent unauthorized or modified code from being run by ensuring that all code is checked before it is executed. The first image in this "chain of trust" is called the Primary Boot Loader (PBL). The Primary Boot Loader is stored in immutable read-only memory. The user can have confidence that this image has not been altered because it cannot be physically altered. This initial software image cryptographically verifies digital signatures on the images that it loads. Those images cryptographically verify the digital signatures on the next set of images that they load, and so on. Hence, the user can have confidence that these images have not been altered because, in every case, a piece of trusted software checks the new image before it can be executed.

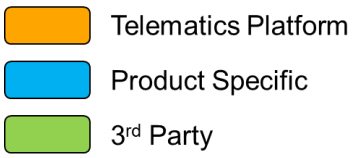
This document provides usage information for enabling Secure Boot feature for ConMod SA515M platform.

1.1 Notational Conventions

Ctrl+x	*A sequence with a '+' indicates that you must press and hold one key or pointing device button while pressing another key or pointing device button*
[ ]	*Optional elements in a command*
…..	*Indicates one of the following possibilities:* • *Additional optional arguments in a statement have been omitted* • *The preceding item or items can be repeated one or more times* • *Additional parameters, values, or other information can be entered*
-	*Line continuation: indicates that a command line or code line continues on the following line when at the end of a line*
Italic text	*Indicates important information, complete titles of manuals or variables*
`Mono-spaced`	*Example code or interactive screen display/output*
Numbers	*All numbers are assumed to be decimal unless otherwise noted. Non decimal radixes – binary, octal or hexadecimal – are explicitly indicated*
Underline blue text	*A link to an external reference (e.g., website, document)*

`Modules`
`|-> section`
`|-> item`
	A link to a section or item in the Telematics Platform html interface specification (Doxygen output from platform code headers)

Diagram Legend



2 Setup

2.1 Tools/Equipment Needed

Sectools: The Sectools is a standalone signing tool that is developed in Python. The sdx55 chipset uses Sectools v5.37. It is used to sign images and generate image for blowing the fuses.

2.2 Setup Instructions

Setup the SDK workspace and enable compilation of linux source code, adding continental's patches and kernel module compilation using the SDK environment following SDK User Guide (TP_F101U-00035).

© Continental Automotive GmbH 2020
Confidential and Proprietary

2.3 Setup Troubleshooting

Not Applicable

3 Features/Components

3.1 Image signing

3.1.1 Overview

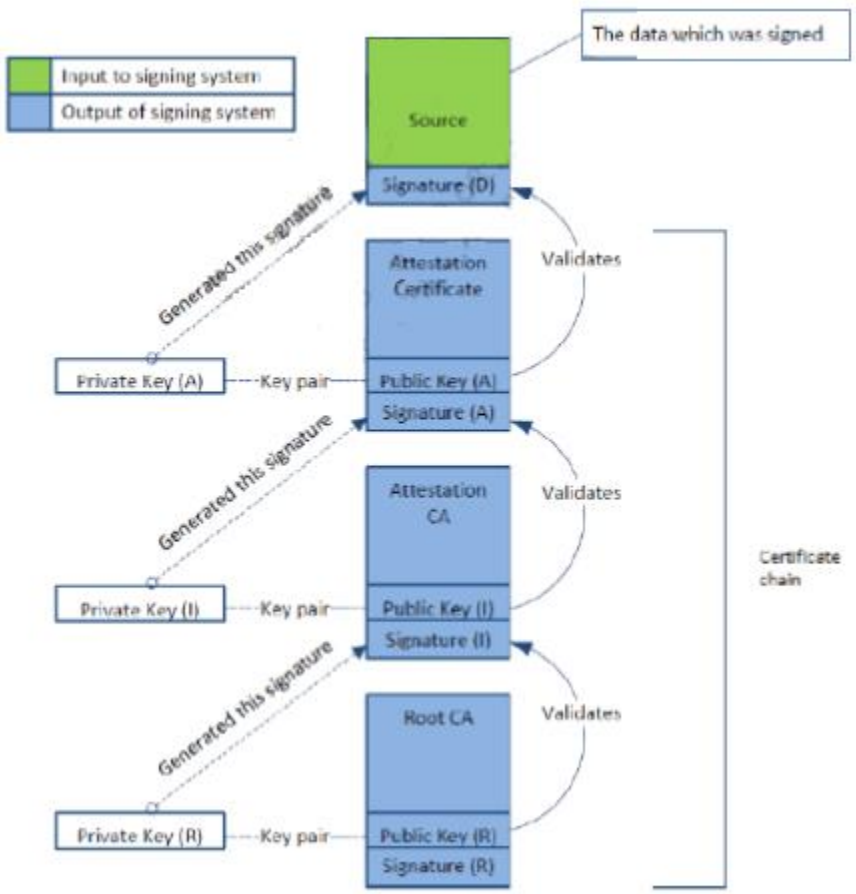
3.1.1.1 Platform Content Overview

3.1.1.1.1 Public Key Infrastructure

Three certificates (*RootCA*, *attesttionCA*, and *attestation*) are used to sign and authenticate Images (except `boot.img`, `boot_tier1.img` and filesystem). *RootCA* and *attesttionCA* certificate are predefined. Three signatures are used for image authentication:

1. one computed over the image source
2. one computed over the attestation certificate
3. one computed over the attestation CA

The root signature itself is validated automatically when the entire root certificate is validated against the provisioned root certificate hash. The certificate chain is depicted below:



© Continental Automotive GmbH 2020
Confidential and Proprietary

Note: `system.ubifs`, `boot.img` and `boot_tier1.img` images do not follow the above certificate chain.

3.1.1.1.2 Image Signing

Below is the supported mode of Image Signing operation

1. **Local:** In this mode, images are signed with the PKI maintained in local build host.
2. **External:** In this mode, images are signed in OEM trusted environment which are online. Signature returns to build server synchronously. Example - Online Trust Center (OTC)

Images signed by continental	Images signed by Others
1. sbl1.mbn 2. tz.mbn 3. devcfg.mbn 4. aop.mbn 5. qdsp6sw.mbn 6. mcfg_hw.mbn 7. multi_image.mbn 8. mcfg_sw.mbn 9. abl.elf 10. uefi.elf 11. multi_image.mbn 12. hyp.mbn 13. prog_firehose_sdx55.mbn 14. ipa_fws.elf 15. cmnlib.mbn 16. tpsecapp32.mbn 17. sec.elf 18. km4.mbn 19. haventkn.mbn 20. boot.img (Linux kernel with embedded Conti certificate) 21. boot_tier1.img (Linux kernel with embedded tier 1 certificate) 22. dsp2.mbn 23. system.ubifs (root filesystem)	1. apdp.mbn (debugpolicy image) 2. system.ubifs (root filesystem) 3. custom Trusted App (optional)

3.1.1.2 Product Expectations

Provide a standalone implementation that receives signing id string and paths to the image hash and to the output signature.

3.1.1.3 Extending Platform Functionality

Not Applicable

3.1.2 Setup

3.1.2.1 Tools/Equipment Needed

Not Applicable

3.1.2.2 Setup Instructions

Not Applicable

3.1.2.3 Setup Troubleshooting

Not Applicable

© Continental Automotive GmbH 2020
Confidential and Proprietary

3.1.3 Usage

3.1.3.1 Basic Instructions

Debugpolicy and filesystem images need to be signed.

3.1.3.2 Detailed Instructions

Signing of debugpolicy and filesystem images are performed via an external image signing adapter which is a python script. The platform provides a reference implementation of the script with skeleton code. The script is located in:

`/.build/scripts/conti_sign_adapter.py`

The script `conti_sign_adapter.py` defines the `contiSignAdapter` class. This class has two methods that must be implemented to sign `apdp.mbn` and `ubifs` images.

User of this guide is responsible for:

- Creating the image signing adapter at the path `/.build/scripts/conti_sign_adapter.py`
- Define the class *contiSignAdapter*
- Implement the method `sign_bootloader` for signing debugpolicy (`apdp.mbn`).
- Implement the method `sign_ubifs` for signing filesystem images.
- (Optional) Implement method `sign_cust_ta` for signing custom trusted application if needed.
- Provide a standalone implementation that receives signing id string and paths to the image hash and to the output signature (See main method in reference implementation).
- The reference Conti adapter script implementation uses the certs embedded as part of the SDK but SDK user shall modify the implementation to access of the keys and store them securely as needed by project. Storing and access of key from adapter is the SDK user or project team responsibility
- All certificates (rootCA, attestationCA and attestation of debugpolicy) shall be in X.509/ASN.1 format and shall be PEM encoded
- The hashing algorithm is SHA256WithRSAPSS with PSS padding scheme (EMSA-PSS as described in PKCS#1 v2.1)

3.1.3.2.1 Steps to sign filesystem (system.ubifs) image:

1. Setup the SDK environment following SDK User Guide (TP F101U-00035).
2. In order to sign `system.ubifs` execute following command:

   `.launchers/linux/sdk/install-prodfs-sdk <workarea_path> <image_size>`
3. Signed filesystem image system.ubifs.sig is located in `<workarea>/release/images/devel/4K/` folder

3.1.3.2.2 Steps to sign debugpolicy (apdp.mbn) image:

1. Setup the SDK environment following SDK User Guide (TP F101U-00035).
2. Change debugpolicy configuration to re-enable ram dump, JTAG, and Trusted logs in `/.build/config/secbootcfg/debugpolicy/sdx55_debugpolicy.xml` (see section 3.3).
3. Recreate and re-sign `apdp.mbn`:

   `.launchers/linux/sdk/install-prodfs-sdk <workarea_path> <image_size>`
4. Signed debugpolicy image `apdp.mbn` can be found in `<workarea>/release/images/devel/4K/` folder

Notes:

- `<workarea_path>` is the absolute path to the place where the sdk-constructor was unpacked.
- `<image_size>` is “4K”.

© Continental Automotive GmbH 2020
Confidential and Proprietary

3.1.3.3 Testing/Error Reporting

Not Applicable

3.2 Enable Secure Boot

3.2.1 Overview

3.2.1.1 Platform Content Overview

Secure boot is enabled through a set of hardware fuses. For the code to be executed, it must be signed by the trusted entity identified in the hardware fuses. By default, images will be signed using pre-generated certificates and keys provided by platform. To enable SecureBoot, `sec.elf` must be flashed onto `sec` partition and **re-install persistency** on the next reboot cycle.

3.2.1.2 Product Expectations

Fuses should be programmed by flashing fuse blower image `sec.elf`.

3.2.1.3 Extending Platform Functionality

Not Applicable

3.2.2 Setup

3.2.2.1 Tools/Equipment Needed

fastboot tool version 1.4.3.

3.2.2.2 Setup Instructions

Not Applicable

3.2.2.3 Setup Troubleshooting

Not Applicable

3.2.3 Usage

3.2.3.1 Basic Instructions

Not Applicable

3.2.3.2 Detailed Instructions

Fuse blower configuration files are placed here:

```
/.build/config/secbootcfg/fuseblower/sdx55_fuseblower_USER.xml
```

3.2.3.2.1 Steps to generate fuse blower image

1. Fuse blower image (`sec.elf`) gets generated as part of command "`bash .launchers/linux/sdk/install-prodfs-sdk`".
2. `sec.elf` will be available in folder `release/images/devel/4K/` as `sec.elf`.

3.2.3.2.2 Steps to flash fuse blower image

Fastboot mode (requires NAD USB):

```
sys_reboot bootloader #takes the target into fastboot mode

fastboot flash sec <Path of sec.elf>
```


© Continental Automotive GmbH 2020
Confidential and Proprietary

Application mode:

Mount the image containing sec.elf in the NAD (Please refer 5G NAD Module User Guide (TP_F103U-00034) - Flashing support over ethernet)

Once image is mounted, please erase the mtd device with sec partition – mtd7 and then program it.

```
flash_eraseall /dev/mtd7
nandwrite -p /dev/mtd7 <Path of sec.elf in target (local or  remote mounted)>
```

Note: Persistency will be erased after this process and needs to be re-flashed.

3.2.3.3 Testing/Error Reporting

Not Applicable

3.3 Suspend Secure Boot

3.3.1 Overview

3.3.1.1 Platform Content Overview

Suspend the secure boot sequence starting from APPSBOOT. This option allows loading an unsigned application and kernel image.

3.3.1.2 Product Expectations

When secure boot is enabled and signed image is flashed, the Debug Policy image with the correct fields should be flashed on the target.

3.3.2 Setup

3.3.2.1 Tools/Equipment Needed

fastboot tool version 1.4.3.

3.3.2.2 Setup Instructions

Not Applicable

3.3.2.3 Setup Troubleshooting

Not Applicable

3.3.3 Usage

3.3.3.1 Basic Instructions

Debug policy configuration file should be updated with appropriate serial number. Configuration file is /.build/config/secbootcfg/debugpolicy/sdx55_debugpolicy.xml.It is possible to specify up to 200 unique serial numbers in this list field. All of the corresponding devices are then supported by the Suspend Secure Boot.

```
<serial_num_list> <!--Range: 1 to 200-->
    <serial_num>0x2466e4ca</serial_num>
    <serial_num>0x2466e4cb</serial_num>
</serial_num_list>
```

Modify parameters DP_ENABLE_SUSPEND_SECUREBOOT_OEM value to 1

```
<flag> <bit_pos>60</bit_pos> <value>0</value> </flag> <!--DP_ENABLE_SUSPEND_SECUREBOOT_OEM-->
```

Create image apdp.mbn and flash to enable Suspend secure boot

© Continental Automotive GmbH 2020
Confidential and Proprietary

3.3.3.2 Detailed Instructions

3.3.3.2.1 Steps to enable Suspend Secure Boot

1. The device serial number can be found from output of "adb devices" command, which can be used to update the serial number list field.
2. serial_num need to be added in list in sdx55_debugpolicy.xml.
3. Set value DP_ENABLE_SUSPEND_SECUREBOOT_OEM to 1
4. Make image using the following command:
   `.launchers/linux/sdk/install-prodfs-sdk <workarea_path> <image_size>`
   `<workarea_path>` is the absolute path to the place where the SDK was installed, `<image_size>` is "4K".
5. Copy signed image `apdp.mbn` from the following folder `/release/images/devel/4K`.
6. Flash `apdp.mbn` either via fastboot or software download package:

   Use fastboot to flash `apdp.mbn` as below:

```
fastboot flash apdp apdp.mbn
fastboot reboot
```

3.3.3.2.2 Steps to disable Suspend Secure Boot

1. Use fastboot to erase apdp as below:

```
fastboot erase apdp
fastboot reboot
```

2. Use a file with 0xFF to overwrite the contents of apdp partition via Software Loading:
   On a Linux environment generate the file to erase apdp partition contents

```
for i in {1..512}; do echo -n -e '\xFF\xFF\xFF\xFF\xFF\xFF\xFF\xFF' >> empty_apdp.mbn; done
```

3.3.3.3 Testing/Error Reporting

Not Applicable

3.4 Re-enabling of ram dump, JTAG and Trusted logs

3.4.1 Overview

3.4.1.1 Platform Content Overview

The Debug Policy images provide a means to enable debugging on commercial secure devices.

3.4.1.2 Product Expectations

When secure boot is enabled, OEMs must blow the debug disable fuses to prevent access to the processors of the device through the JTAG ports.

This action is essential to ensure that secure boot cannot be bypassed. Debug policy images are used to enable debugging on secure boot enabled devices.

- DebugPolicy xml file is used to configure, validate, and generate debugpolicy image (apdp.mbn), which enables debugging of images on target:
- debugpolicy image(apdp.mbn) need to be signed (see section 3.1.3.2.3).
- For a specific device, the serial number of the device can be set.
- For a number of devices list of serial numbers can be set to allow multiple devices to be supported.
- Flags can be provided to re-enable e.g. JTAG, crash dumps, trusted logs, etc.

© Continental Automotive GmbH 2020
Confidential and Proprietary

3.4.1.3 Extending Platform Functionality

Not Applicable

3.4.2 Setup

3.4.2.1 Tools/Equipment Needed

fastboot tool version 1.4.3.

3.4.2.2 Setup Instructions

Not Applicable

3.4.2.3 Setup Troubleshooting

Not Applicable

3.4.3 Usage

3.4.3.1 Basic Instructions

Create image `apdp.mbn` and flash to enable ram dump, JTAG and Trusted logs on a secure target.

3.4.3.2 Detailed Instructions

Debug Policy image is `apdp.mbn` in folder `/release/images/devel/4K/apdp.mbn`.

Flash Debug Policy image on to the target:

```
fastboot flash apdp <apdp.mbn file with path>
```

Configuration file is `/.build/config/secbootcfg/debugpolicy/sdx55_debugpolicy.xml`

```
<flags>
    <flag> <bit_pos>0</bit_pos> <value>0</value> </flag> <!--DP_ENABLE_ONLINE_CRASH_DUMPS-->
    <flag> <bit_pos>1</bit_pos> <value>0</value> </flag> <!--DP_ENABLE_OFFLINE_CRASH_DUMPS-->
    <flag> <bit_pos>2</bit_pos> <value>0</value> </flag> <!--DP_ENABLE_JTAG-->
    <flag> <bit_pos>3</bit_pos> <value>0</value> </flag> <!--DP_ENABLE_LOGS-->
    <flag> <bit_pos>4</bit_pos> <value>0</value> </flag> <!-- DP_ENABLE_MODEM_INV_DEBUG -->
    <flag> <bit_pos>5</bit_pos> <value>0</value> </flag> <!-- DP_ENABLE_MODEM_NINV_DEBUG -->
    <flag> <bit_pos>6</bit_pos> <value>0</value> </flag> <!-- DP_ENABLE_DEBUG_INV -->
    <flag> <bit_pos>7</bit_pos> <value>0</value> </flag> <!-- DP_ENABLE_DEBUG_LEVEL -->
    <flag> <bit_pos>8</bit_pos> <value>0</value> </flag> <!-- DP_ENABLE_DEBUG_LEVEL -->
    <flag> <bit_pos>24</bit_pos> <value>1</value> </flag> <!--DP_ENABLE_NONSECURE_CRASH_DUMPS-->
    <flag> <bit_pos>25</bit_pos> <value>0</value> </flag> <!--DP_ENABLE_APPS_ENCRYPTED_MINI_DUMPS-->
    <flag> <bit_pos>26</bit_pos> <value>0</value> </flag> <!--DP_ENABLE_MPSS_ENCRYPTED_MINI_DUMPS-->
    <flag> <bit_pos>27</bit_pos> <value>0</value> </flag> <!--DP_ENABLE_LPASS_ENCRYPTED_MINI_DUMPS-->
    <flag> <bit_pos>28</bit_pos> <value>0</value> </flag> <!--DP_ENABLE_CSS_ENCRYPTED_MINI_DUMPS-->
    <flag> <bit_pos>29</bit_pos> <value>0</value> </flag> <!--DP_ENABLE_ADSP_ENCRYPTED_MINI_DUMPS-->
    <flag> <bit_pos>30</bit_pos> <value>0</value> </flag> <!--DP_ENABLE_CDSP_ENCRYPTED_MINI_DUMPS-->
</flags>

<image_bit_map> <!--Range: 0 to 32-->
    <swid> <bit_pos>0</bit_pos> <value>0</value> </swid> <!--XBL-->
    <swid> <bit_pos>1</bit_pos> <value>0</value> </swid> <!--MBA-->
    <swid> <bit_pos>2</bit_pos> <value>0</value> </swid> <!--AMSS_HASH_TABLE-->
</image_bit_map>

<root_cert_hash_list> <!--Range: 0 to 4-->
    <root_cert_hash>0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000</root_cert_hash>
</root_cert_hash_list>

<serial_num_list> <!--Range: 1 to 200-->
    <!--<serial_num>00000000</serial_num>-->
</serial_num_list>

<root_cert_hash_qti_list> <!--Range: 0 to 4-->
    <root_cert_hash_qti>0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000</root_cert_hash_qti>
</root_cert_hash_qti_list>
```

`DP_ENABLE_NONSECURE_CRASH_DUMPS` flag:

- 0: ram dump not allowed on secure target

© Continental Automotive GmbH 2020
Confidential and Proprietary

- 1 (default): ram dump allowed on secure target

`DP_ENABLE_JTAG` flag:

- 0 (default): JTAG disabled on secure target
- 1: JTAG enabled on secure target

`DP_ENABLE_LOGS` flag:

- 0 (default): trusted logs disabled on secure target
- 1: if trusted logs enabled on secure target

`DP_ENABLE_NONSECURE_CRASH_DUMPS` does not allow subsystem modem ramdump. It requires `DP_ENABLE_ONLINE_CRASH_DUMPS`, `DP_ENABLE_OFFLINE_CRASH_DUMPS` with a list of `serial_num`, (max 200). For subsystem ramdump the serial number list should be updated in debugpolicy configuration at `/.build/config/secbootcfg/debugpolicy/sdx55_debugpolicy.xml`. It is possible to specify up to 200 unique serial numbers in this list field. All of the corresponding devices are then supported by the debug policy and subsystem ramdump will work.

```
<serial_num_list> <!--Range: 1 to 200-->
    <serial_num>0x2466e4ca</serial_num>
    <serial_num>0x2466e4cb</serial_num>
</serial_num_list>
```

`DP_ENABLE_NONSECURE_CRASH_DUMPS` is not restricted to serial number list. When ram dumps are enabled via this flag, subsystem application processor ram dumps can be collected on any TCU. It is recommended to have `DP_ENABLE_NONSECURE_CRASH_DUMPS` set to disabled (0) and only set to enabled (1) when required. All flags other than `DP_ENABLE_NONSECURE_CRASH_DUMPS` are restricted by TCU serial number, which must be in the serial number list (See Known Issues).

3.4.3.2.1 Steps to enable subsystem ramdump

1. Set `DP_ENABLE_ONLINE_CRASH_DUMPS`, `DP_ENABLE_OFFLINE_CRASH_DUMPS` flags in `/.build/config/secbootcfg/debugpolicy/sdx55_debugpolicy.xml`.
2. The device serial number can be found from output of "adb devices" command, which can be used to update the serial number list field to enable susbsystem ramdump.
   ```
   D:\>fastboot devices
   2466e4ca        fastboot
   ```
3. `serial_num` need to be added in list in `sdx55_debugpolicy.xml`.
4. Make image using the following command:
   ```
   bash .launchers/linux/sdk/install-prodfs-sdk
   ```
5. Copy signed image `apdp.mbn` from 4K image folder.
6. Flash `apdp.mbn` either via fastboot or software download package:
   a. Use fastboot to flash `apdp.mbn` as below:
      ```
      fastboot flash apdp apdp.mbn
      fastboot reboot
      ```
   b. Follow Update Package Generation User Guide (TP_F101U-00040) to flash `apdp.mbn` from software download package.

See Update 5G NAD Module User Guide (TP_F103U-00034) for additional information on collecting the ramdump.

3.4.3.2.2 Steps to disable Debug policy

1. Use fastboot to erase apdp as below:
   ```
   fastboot erase apdp
   fastboot reboot
   ```


© Continental Automotive GmbH 2020
Confidential and Proprietary

2. Use a file with 0xFF to overwrite the contents of apdp partition via Software Loading:
    a. On a Linux environment generate the file to erase apdp partition contents
    `for i in {1..512}; do echo -n -e '\xFF\xFF\xFF\xFF\xFF\xFF\xFF\xFF' >> empty_apdp.mbn; done`
    b. Include `empty_apdp.mbn` file in a software download package by following Update Package Generation User Guide TP_F101U-00040

3.4.3.3 Testing/Error Reporting

Not Applicable

4 Known Issues

- sectools does not support RSA/PKCS signing.
- Anti-rollback of `boot.img` and Filesystem not supported.
- `DP_ENABLE_NONSECURE_CRASH_DUMPS` does not allow subsystem modem ramdump
  (`DP_ENABLE_ONLINE_CRASH_DUMPS`, `DP_ENABLE_OFFLINE_CRASH_DUMPS` are required, providing a list of serial_num, maximum 200)

5 FAQs

Not Applicable

6 Troubleshooting/Tips and Tricks

Not Applicable

7 References

7.1 Abbreviations/Acronyms/Definitions

Not Applicable

7.2 Platform Documentation

TP_102G-00001: Telematics Platform Glossary
TP_F103U-00034: 5G NAD Module User Guide
TP_F101U-00035: Software Development Kit User Guide
TP_F101U-00040: Update Package Generation User Guide

7.3 Website links

Not Applicable

© Continental Automotive GmbH 2020
Confidential and Proprietary

8 Revision History

Version	Date	Description
01.00	Sep 3, 2021	Document converted to ConMod project specific
02.00	Aug 9, 2024	Added information regarding custom TA signing

© Continental Automotive GmbH 2020
Confidential and Proprietary